

Updated Quran Recitation AI Architecture

Forced-Alignment and Phoneme Segmentation

Modern forced-aligners match recitation audio to known text without full ASR. Classic tools like **Montreal Forced Aligner (MFA)** (HMM/Kaldi) provide word/phone boundaries and a “phone_goodness” confidence score for each segment. New neural methods leverage self-supervised models: for example, **Wav2Vec2** or **WhisperX**. HuggingFace-torchAudio pipelines (CTC-based) can use a pretrained Wav2Vec2 as the acoustic model for CTC alignment of a given transcript. **WhisperX** (built on OpenAI Whisper) performs full utterance transcription and then aligns the text to audio, optionally yielding subword/phone timings using language-specific models. Research shows Wav2Vec2-based aligners (e.g. Meta’s MMS multilingual model) approach the accuracy of HMM aligners: on a 53-language benchmark Wav2Vec2 (MMS) achieved ~95% phone alignment vs ~96% for Kaldi (English). Libraries like **Charsiu** (GitHub: [lingjzhu/charsiu](https://github.com/lingjzhu/charsiu)) implement Wav2Vec2-based phonetic aligners (forced and predictive modes) for English/Chinese and can be extended to Arabic. **BFA (Bournemouth Forced Aligner)** uses a contextless phoneme encoder and can output phoneme timings with confidence scores (frame-level probability averages). In practice one might pipeline: (1) compute frame-level features via Wav2Vec2, (2) apply a CTC segmentation/forced-alignment (torchAudio or WhisperX), and (3) extract per-word/phone start-end times with confidences for error checking.

- **Key tools/models:** MFA (HMM/Kaldi); Kaldi/Gentle; HuggingFace Wav2Vec2 ASR pipelines; Whisper/WhisperX with CTC segmentation; Charsiu/BFA (Wav2Vec2 phoneme aligners).
- **Outputs:** Each word/phoneme interval with start/end times and a score or confidence (e.g. MFCC/Viterbi log-likelihood). MFA labels each phone with a “phone_goodness” score. Neural aligners can similarly output averaged log-probabilities or posterior scores for each phoneme, enabling low-confidence filtering.

DTW-based Recitation Alignment

Once phoneme timings are obtained, we align the student’s recitation to a reference Sheikh’s recitation using **Dynamic Time Warping (DTW)** on acoustic features. For example, compute MFCC or mel-spectrogram frames for both recordings and run DTW to find the optimal time warp path. Libraries like **librosa** provide `librosa.sequence.dtw`, which aligns two feature matrices via optimal-path search. (Alternatively, **tslearn** offers Soft-DTW for time series.) The figure below illustrates how DTW warps two time-series to align similar content ¹.

Fig. 1. Aligning two sequences via Dynamic Time Warping. In practice we extract features (e.g. mel-spectrogram or MFCC frames) for student vs. expert audio and run DTW to align phoneme or syllable boundaries ¹.

Key libraries for DTW alignment: **librosa** (`librosa.sequence.dtw`), **tslearn** (DTW/Soft-DTW), or simple custom DTW implementations. TorchAudio does not natively expose DTW, but one can use PyTorch to compute distance matrices and path if needed. The DTW alignment yields, for each phoneme in one audio,

the matching time interval in the other audio, allowing direct duration and prosody comparison across recitations.

Phoneme-Level Feature Extraction (Duration, Pitch, Energy)

Given phoneme segmentation, compute acoustic features per segment. **Duration** is simply `end_time - start_time` for each phoneme or letter. For **pitch/F0**, tools like **Librosa** or **Praat/Parselmouth** can extract fundamental frequency tracks. For instance, `librosa.pyin(y, fmin, fmax)` computes the probabilistic YIN F0 estimate. **Energy or intensity** can be obtained via short-term energy (e.g. `librosa.feature.rms()`) or amplitude. Specialized toolkits like **Shennong** (Python) offer a suite of feature extractors (MFCC, filterbanks, neural features, and pitch estimators) built on Kaldi. An approach is: for each aligned phoneme interval, crop the audio and compute (a) **median F0** from `pyin`, (b) **RMS energy**, (c) **spectral centroid** or **formant** frequencies if needed (Parselmouth can yield formants). These numeric features then feed into error rules. For example, a very low pitch or low energy on an expected Qalqalah might signal an under-pronounced plosive. Tools: **Librosa** (features: `pyin`, `piptrack`, `rms`, etc.), **Praat (via Parselmouth)**, **PyWorld** for high-quality pitch, **openSMILE** for prosody analysis. Shennong's documentation specifically notes support for pitch estimation and filterbank features, making it a candidate library.

- **Example:** Use `f0, voiced_flag, voiced_prob = librosa.pyin(y_segment, fmin=65, fmax=2093)` to get the pitch contour; use `librosa.feature.rms(y=y_segment)` for energy. Normalize durations by comparing to the reference duration from DTW alignment.

Tajweed Rule Error Detection (Madd, Ghunnah, Qalqalah)

After alignment, detect specific Tajweed rule violations via acoustic patterns:

- **Madd (elongation):** Madd involves lengthening certain vowels or letters (ي, و, ا, or end-of-ayah vowels) to 2, 4, or 6 "harakat" (time units). Acoustically, this means the vowel's duration in student audio greatly exceeds the reference vowel's duration (often by a factor of 2–3×). One can simply compare aligned phoneme durations: if `duration_student / duration_reference > threshold` (e.g. 2× or 3×), flag a Madd error. Shafie et al. (2022) align syllable-level STFT features by DTW and use a threshold on the minimum path cost to classify Madd violations. In our pipeline, we can set a duration-ratio threshold (inspired by the "2,4,6" rule lengths) to classify a Madd error.
- **Ghunnah (nasalization):** Ghunnah is the nasal resonance on /m/ or /n/ in specific contexts, held for roughly two counts. It's characterized by extra nasal formant energy. Acoustically, a correctly pronounced Ghunnah will show lowered amplitude of F1 (nasalization) and possibly a nasal formant peak around 250 Hz. Meftah & Alotaibi (2015) classify nasalized vs. non-nasal Quranic phonemes using the first three formants. In practice, one could compute formant magnitudes (Praat) or spectral tilt and check if nasalization features are present during the /m/ or /n/ interval. A simple heuristic: if the segment is labeled nasal but the energy in the lower formant band is abnormally low (or formant shift is absent) relative to a non-nasal reference, flag a Ghunnah error.
- **Qalqalah (echoing plosive):** Qalqalah affects the letters ق, ط, ب, ج, د with a "bounce" or echo on stop closure. Acoustically this appears as an abrupt burst of high-frequency energy when the stop is

released. Mazid & Ahmad's "TajweedAI" (2025) specifically targets Qalqalah: after ASR-based segmentation they extract features (MFCCs, spectral centroid, zero-crossing rate) and train a classifier to detect the characteristic "echo". Practically, one can compute the local energy rise: e.g. measure the spectrum just before vs. after the expected stop release. A high spectral centroid or energy spike on the stop indicates correct Qalqalah. If a weak burst (low centroid, low high-frequency energy) is detected, the system flags an error. The AI study achieved very high accuracy by such focused features.

For **feedback generation**, we set empirical thresholds on these features (durations, nasal energy, burst energy). For example, if Madd ratio $>2.5\times$, output "Extend the vowel longer (Madd)"; if nasal-energy $<$ threshold, say "Add more nasalization (Ghunnah)". Prior work often uses fixed error thresholds: e.g. Shafie et al. use a minimum-path-cost threshold for Madd, and BFA suggests discarding phonemes with averaged confidence below a threshold. We can similarly threshold confidence or feature differences and then generate templated suggestions (e.g. "Pronounce the *Ghunnah* with your nose," "Emphasize the Qalqalah burst").

Open Datasets and Prior Systems

There are emerging datasets and systems for Arabic/Quranic speech with annotations:

- **Tajweed Annotation (text):** The `quran-tajweed` project provides comprehensive Tajweed rule annotations on the Arabic text (in JSON), including Ghunnah, Madd (2/4/6), Qalqalah, etc.. This can guide which words to check acoustically.
- **Audio/Time-Stamped Quran:** Tarteel's *Quranic Universal Library* (QUL) offers curated recitation audio with timestamps. It hosts many high-quality recitations along with word- and ayah-level timing metadata ². These can serve as reference ("Sheikh") recordings or as training data.
- **Quran-MD Multimodal Corpus:** The *Quran-MD* dataset (HuggingFace) contains verse-level and word-level Quran audio from 30 reciters with aligned Arabic text, translations, and transliterations ³ ⁴. Its creators explicitly cite tasks like ASR and Tajweed detection as intended uses ³. Word-level recordings could be used to train phoneme-recognition or confidence models.
- **Research on Quran Recitation ASR/Mispronunciation:** Several studies use ML to detect Tajweed errors. For example, Al-Ayyoub et al. (2023) applied MFCC/GMM/MLP pipelines for basic rule detection. Küçükmanisa et al. (2025) built a *multimodal* Arabic phoneme error detector combining Whisper transcription embeddings and Wav2Vec2 acoustic embeddings, achieving $>90\%$ accuracy on 29 phonemes. Mazid & Ahmad (2025) demonstrated a TajweedAI system focusing on Qalqalah with a hybrid ASR (for alignment) + feature classifier. These indicate feasibility of our approach: using pretrained speech models (Whisper, Wav2Vec2) plus classifiers on engineered features.
- **Open Tools:** In addition to the aligners, the community maintains Python libraries that support this architecture. TorchAudio's `align` pipelines (Wav2Vec2 CTC segmentation), Praat's Parselmouth interface for pitch/formants, and the Shennong toolkit are all accessible. There are also ASR toolkits (ESPnet, Kaldi) that, while heavy, can be scripted in Python. Projects like the Kaggle *Tajweed Dataset*

(categorizing audio by rule) or GitHub scripts (e.g. `quran-ai-transcribing`) demonstrate data processing pipelines.

Implementation Recommendations

In practice, a Python-based pipeline might use:

- **Alignment:** Use a HuggingFace Wav2Vec2 model (Arabic ASR or multilingual XLSR) with `torchaudio.functional.forced_align` or WhisperX to align the known Ayah text to user audio and get phoneme timings.
- **Feature Extraction:** Use `librosa` to extract mel or MFCC matrices, and DTW-align them. Compute per-phoneme **duration**, **F0** (via `librosa.pyin`), and **RMS energy**. For nasal features (Ghunnah), use Praat-Parselmouth to get formants. For Qalqalah, compute short-time spectral peak.
- **DTW Alignment:** Apply `librosa.sequence.dtw` on the per-frame features of student vs. expert recitation to get a mapping between their frames. This yields aligned time intervals for each phoneme, allowing direct comparison of duration/pitch.
- **Error Detection:** Compare each matched phoneme: e.g. if `duration_ratio > 2×`, flag Madd; if nasal formant energy is too low, flag Ghunnah; if stop-burst energy is insufficient, flag Qalqalah. Use confidence scores from aligner (and thresholds as in BFA) to ignore low-likelihood phones.
- **Feedback Text:** Predefine error messages (e.g. "Hold the Madd longer", "Add more nasal resonance") triggered by each rule condition.

All chosen libraries are open-source and Python-compatible. The approach is technically feasible: e.g. torchaudio+transformers can handle Arabic speech, librosa+pyroomacoustics handle DTW and features, and datasets like QUL/Quran-MD provide data. Prior works show accuracy of this style (e.g. DTW+threshold for Madd, formant/feature classifiers for Ghunnah/Qalqalah).

References: State-of-the-art alignment and ASR tools; forced-align tutorials; phoneme alignment research ⁵ ; DTW methods ¹ ; feature extraction tools; and specific Tajweed studies ² ³ .

¹ File:Dynamic Time Warping.png - Wikimedia Commons
https://commons.wikimedia.org/wiki/File:Dynamic_Time_Warping.png

² Quranic Universal Library
<https://qul.tarteel.ai/>

³ ⁴ Buraaq/quran-audio-text-dataset · Datasets at Hugging Face
<https://huggingface.co/datasets/Buraaq/quran-audio-text-dataset>

⁵ GitHub - tabahi/bournemouth-forced-aligner: Extract phoneme-level timestamps from speech audio.
<https://github.com/tabahi/bournemouth-forced-aligner>